Data Governance Copilot

Interview Question Bank

Topic 02 — Multi-Agent Design Patterns

12 questions · 4 sections · Supervisor · Specialized Agents · INTENT_AGENT_MAP · Protocols · Scaling

ndational Core concepts | Intermediate Design decisions | Advanced Deep internals | Gotcha Real bugs / traps

---

Supervisor Pattern & System Architecture

Q01 Foundational

Why did you choose a Supervisor + Specialized Agents architecture over a single monolithic LLM agent?

Hint: Think about separation of concerns, accuracy, and maintainability.

Key points to cover:

- A monolithic agent handles all domains in one prompt — context bloat, hallucination risk, harder to test
- Specialized agents are experts in one domain: InformationAgent knows SQL/Databricks, KnowledgeAgent knows RAG, MetadataAgent knows Collibra
- Supervisor handles only orchestration — it classifies intent and routes; it doesn't answer the question itself
- Separation of concerns: each agent can be tested, mocked, and improved independently without touching others
- Failure isolation: if MetadataAgent fails, InformationAgent still runs — partial answers are possible
- Scalability: new business domain → add one agent + one INTENT_AGENT_MAP entry, no changes to existing agents
- Mirrors real-world data governance teams: data engineers (information), knowledge managers (knowledge), catalog owners (metadata)

Q02 Foundational

What is the role of supervisor_agent.py vs the supervisor_node in graph/nodes.py? Why does your project have both?

Hint: One is a legacy stub; the other is the active orchestrator.

Key points to cover:

- supervisor_agent.py is a legacy stub — created early in the project when agents were standalone classes, now superseded
- supervisor_node in nodes.py is the active LangGraph node — it runs as part of the StateGraph execution
- supervisor_node reads AgentState, calls intent classification (get_structured_llm), writes intent + next_agents back to state
- It does NOT call agents directly — it only sets routing instructions; LangGraph's conditional edges do the actual routing
- supervisor_agent.py is kept for backward compatibility (some tests may reference it) but adds no runtime value
- Clean architecture lesson: as projects evolve, orchestration responsibility migrates from imperative code to the graph framework
- Interview angle: demonstrates awareness that the project evolved — shows honest reflection on architectural debt

---

Q03 Intermediate

How does the supervisor know which agents to invoke? Walk through the data flow from query to next_agents list.

Hint: Trace: user query → intent classification → agent list in state.

Key points to cover:

- 1. pre_hook writes query, query_id, start_time to state; guardrails pass
- 2. supervisor_node receives full AgentState with the user's query string
- 3. Builds a classification prompt including query + available data_products from config
- 4. Calls get_structured_llm(config, IntentClassification).invoke(prompt) → IntentClassification Pydantic object
- 5. Calls get_agents_for_intent(intent) → looks up INTENT_AGENT_MAP[intent] → returns list of agent name strings
- 6. Writes intent, data_products, confidence, next_agents to AgentState
- 7. _agent_router conditional edge reads state['next_agents'][0] → routes to that agent node name
- 8. _wrap_agent + _agents_ran tracker ensures sequential execution through the full next_agents list

```
# routing.py INTENT_AGENT_MAP = { 'full_diagnostic':
['information','knowledge','metadata','capacity'], 'data_quality': ['metadata','information'],
'governance': ['metadata','knowledge'], 'write_rule': ['rule'], 'unknown':
['information','knowledge'], # ... 6 more } def get_agents_for_intent(intent: str) -> List[str]:
return INTENT_AGENT_MAP.get(intent, ['information','knowledge'])
```

INTENT_AGENT_MAP — Design Decisions

Q04 Intermediate

Walk through the INTENT_AGENT_MAP. For 'full_diagnostic', why are all four agents in that specific order?

Hint: Agent ordering in sequential routing is deliberate — later agents can use earlier agents' results.

Key points to cover:

- full_diagnostic: ['information','knowledge','metadata','capacity'] — broadest diagnostic, all domains
- information first: raw metrics/data from Databricks — establishes factual baseline (what the numbers say)
- knowledge second: RAG over governance docs — contextualises the numbers against policies and past incidents
- metadata third: Collibra catalog — adds data lineage, ownership, quality scores for the assets in question
- capacity last: Jira incidents — checks if there are open tickets already covering the issue
- Ordering matters for synthesizer: synthesizer_node sees agent_results in execution order — information → knowledge → metadata → capacity provides a natural narrative flow
- If capacity ran first, the synthesizer might emphasise incident tickets before even establishing what the data shows

Q05 Intermediate

Why does 'data_quality' route to ['metadata','information'] but 'metric_analysis' routes to ['information','knowledge']? What is the reasoning behind each?

Hint: Agent selection reflects the nature of each question type.

Key points to cover:

- data_quality: primary source is Collibra (MetadataAgent) — it holds DQ scores, completeness metrics, validation rules
- data_quality secondary: InformationAgent (Databricks SQL) — validates DQ claims with raw row counts, null rates
- metadata leads for data_quality because Collibra IS the system of record for quality — it should anchor the answer
- metric_analysis: primary source is Databricks SQL (InformationAgent) — business KPIs live in the warehouse
- metric_analysis secondary: KnowledgeAgent (RAG) — provides context from governance docs, metric definitions, historical trends
- Collibra is NOT in metric_analysis — catalog metadata doesn't help explain why revenue dropped 12%
- Design principle: always ask 'what is the authoritative source for this question type' — that agent leads

---

Q06 Advanced

What are the write intents (write_ticket, write_metadata, write_rule) and why do they each route to a single agent? What guardrails protect them?

Hint: Write operations are a different category from read/analysis intents.

Key points to cover:

- write_ticket → ['capacity']: only CapacityAgent talks to Jira — single-agent prevents duplicate ticket creation
- write_metadata → ['metadata']: only MetadataAgent has write access to Collibra — prevents conflicting catalog updates
- write_rule → ['rule']: RuleAgent manages the rule registry exclusively — no other agent should mutate rules
- Single agent for writes: eliminates race conditions (two agents trying to create the same Jira ticket)
- Guardrails stack: pre_hook blocks SQL injection and prompt injection before the graph even reaches routing
- HITL gate: auto_ticket_node checks for write actions with anomalies — requires human approval before executing
- Confidence threshold: if intent confidence < 0.6 on a write intent, consider requiring explicit confirmation
- Audit trail: every write action goes through post_hook → execution_ms logged; Collibra/Jira have their own audit logs

---

Q07 Advanced

A new use case arrives: 'Show me the lineage of the bookings table and create a governance incident if there are gaps.' Which intent does this map to and how would you handle it?

Hint: This tests whether you can extend the system for compound intents.

Key points to cover:

- This is a compound intent: 'governance' (lineage from Collibra) + 'write_ticket' (Jira incident creation)
- Current INTENT_AGENT_MAP doesn't have a compound write+read intent — 'governance' only routes to ['metadata','knowledge']
- Option 1: add 'governance_incident' intent → ['metadata','knowledge','capacity'] — capacity handles the Jira write
- Option 2: two-pass execution — first run 'governance' intent, then chain 'write_ticket' if lineage gaps found
- Option 3: extend CapacityAgent to be included in governance routing when write keywords are detected
- HITL integration: lineage gap auto-creates a pending_action; HITL approval triggers capacity_node to create the Jira ticket
- Best answer: add a new intent 'governance_incident' with routing ['metadata','knowledge','capacity'] + HITL gate
- Demonstrates extensibility: the system is designed to add intents without changing existing agent code

---

Agent Protocol Design — BaseAgent & Services

Q08 Foundational

Explain the BaseAgent contract. What is AgentRequest, what is AgentResult, and why are they Pydantic models?

Hint: Think about type safety across agent boundaries.

Key points to cover:

- BaseAgent is an abstract class with a single abstract method: execute(request: AgentRequest) -> AgentResult
- AgentRequest carries: query, thread_id, user_id, data_products, time_range, context — everything an agent needs
- AgentResult carries: success (bool), data (dict), sources (list), error (str|None), execution_ms (float)
- Pydantic models enforce type safety: an agent can't accidentally return raw dicts or skip required fields
- success: bool is the canonical health signal — nodes check result.success before appending to agent_results
- sources: List[str] is accumulated across agents in state — synthesizer uses this for citation in final_summary
- error: Optional[str] is set on failure — gets appended to state['errors'] for debugging without crashing the graph
- Pydantic V2 also enables .model_dump() for JSON serialization — important for Redis cache storage

```
class AgentRequest(BaseModel): query: str thread_id: str user_id: str = 'anonymous' data_products: List[str] = [] time_range: str = '30d' context: dict = {} class AgentResult(BaseModel): success: bool data: dict = {} sources: List[str] = [] error: Optional[str] = None execution_ms: float = 0.0
```

Q09 Intermediate

How does the services layer decouple agents from their external dependencies? Walk through the IDataService protocol.

Hint: This is the dependency injection / service abstraction pattern.

Key points to cover:

- IDataService is a runtime_checkable Protocol — defines the interface without inheritance
- Protocol methods: query_metrics(sql, params) → dict, get_data_products() → List[str]
- InformationAgent receives an IDataService in its constructor — it never imports DatabricksService directly
- In prod (ENABLE_MOCK=false): factory.get_data_service() returns DatabricksService(real SQL warehouse)
- In dev/test (ENABLE_MOCK=true): factory returns MockDatabricksService(canned responses, no network)
- runtime_checkable: isinstance(service, IDataService) works at runtime — factory can validate injected service
- Benefit: InformationAgent tests run without Databricks credentials — pytest can inject MockDatabricksService
- Four protocols total: IDataService, ITicketService, IMetadataService, IVectorService — one per external system

```
# Protocol definition class IDataService(Protocol): def query_metrics(self, sql: str, params: dict) -> dict: ... def get_data_products(self) -> List[str]: ... # Agent constructor class InformationAgent(BaseAgent): def __init__(self, service: IDataService): self.service = service # real or mock, agent doesn't care # Factory switching def get_data_service() -> IDataService: if config.enable_mock: return MockDatabricksService() return DatabricksService(config.databricks)
```

Q10 Intermediate

The RuleAgent is different from the other four agents — it doesn't have a real/mock service pair. Why, and what does it do instead?

Hint: Think about what the rule registry actually is.

Key points to cover:

- RuleAgent manages the internal rule registry — it doesn't call any external third-party system
- The rule registry is stored directly in the application database (Neon PostgreSQL / governance_rules table)
- Operations: CRUD (create, read, update, delete rules) + evaluate(rule_id, data) — pure business logic
- No external dependency means no service protocol needed — RuleAgent talks directly to the DB via psycopg2 or SQLAlchemy
- This is intentional: governance rules are the system's own knowledge, not sourced from Databricks or Collibra
- write_rule intent routes exclusively here — only RuleAgent has authority to mutate the rule registry
- In tests: patch the DB connection (psycopg2.connect) rather than a service — same module-level import discipline applies
- Analogy: RuleAgent is the system's internal policy engine; MetadataAgent is the external catalog interface

---

Tricky / Design & Gotcha Questions

Q11 Gotcha

Two agents both write to state['sources'] (an Annotated list). Agent A returns ['collibra://asset/123'] and Agent B returns ['databricks://table/bookings']. What does state['sources'] contain after both run?

Hint: Think about how operator.add works on lists across sequential node executions.

Key points to cover:

- sources: Annotated[List[str], operator.add] — operator.add concatenates lists from each node's output
- After InformationAgent: state['sources'] = ['databricks://table/bookings']
- After MetadataAgent: LangGraph calls operator.add(['databricks://table/bookings'], ['collibra://asset/123'])
- Final state['sources'] = ['databricks://table/bookings', 'collibra://asset/123'] — both preserved
- This is why Annotated + operator.add is critical: without it, MetadataAgent's sources would overwrite InformationAgent's
- synthesizer_node reads all sources and can include them as citations in final_summary
- Deduplication: if two agents both reference the same source, it appears twice — consider adding a dedup step in post_hook
- Order: sources appear in agent execution order — reflects the sequential routing sequence defined in next_agents

---

Q12 Design

You've built 5 specialized agents. An interviewer asks: 'How would you scale this to 20 agents without the routing becoming unmaintainable?' What is your answer?

Hint: This tests architectural thinking beyond your current implementation.

Key points to cover:

- Current INTENT_AGENT_MAP is a static dict — 20 agents with 15+ intents means 300 possible combinations to maintain
- Solution 1: hierarchical routing — supervisor routes to domain coordinators (Data Domain, Governance Domain, Ops Domain), each coordinator routes to its own sub-agents
- Solution 2: capability tags — each agent declares capabilities (['sql', 'metrics', 'databricks']); supervisor queries 'which agents have capability X' rather than hard-coded map
- Solution 3: LLM-based dynamic routing — supervisor asks the LLM which agents are needed given the query and agent capability descriptions (AgentCard pattern from Google A2A protocol)
- Solution 4: load-based routing — if InformationAgent is overloaded, route to a replica or skip with graceful degradation
- Retain static map for write intents — safety-critical operations should never be dynamically routed
- Your current design scales well to ~8-10 agents with static map; beyond that, capability-tag routing is the right evolution
- Mention: this mirrors how Anthropic's multi-agent research describes 'orchestrator + subagent' patterns at scale